Pipelining

1 Introduction and Pipeline Concepts

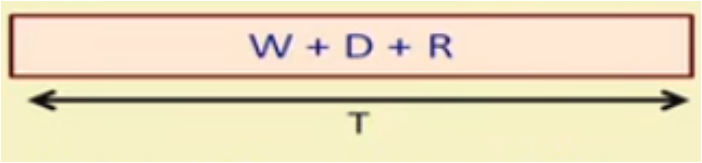
1.1 What is pipelining?

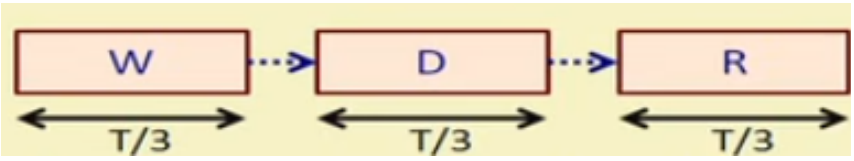
Pipelining overlaps the execution of multiple input sets by partitioning a computation into a sequence of smaller stages. It can improve throughput with only a modest increase in implementation cost. Each input still passes through all stages, but several inputs can be in progress at once.

Common applications include instruction execution, where different instructions occupy different stages at the same time; arithmetic computation, where the same operation is applied to successive data sets; and memory access, where successive locations are accessed in sequence.

1.2 A real-life analogy

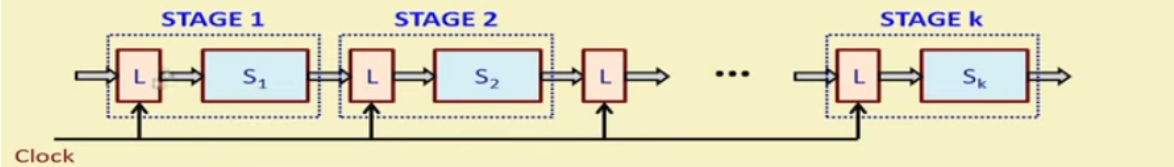
Suppose one machine washes (W), dries (D), and irons (R) one item at a time, with total processing time T per item. Processing N items sequentially takes $T_1 = NT$. If the work is split among three machines, each taking about $T/3$, the stages can overlap: while one item is dried, another can be washed and an earlier item ironed. The total time becomes $T_3 = (N + 2)T/3$, including the time to fill the three-stage pipeline.



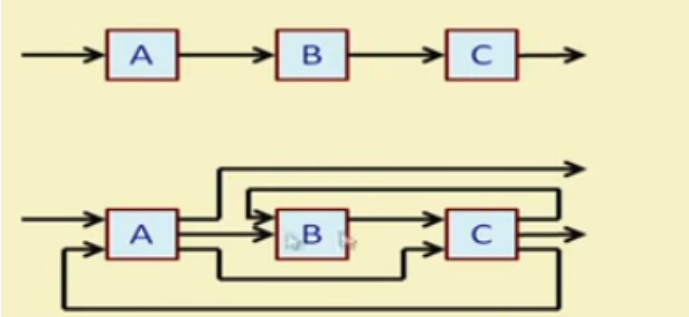


The same choices apply to a processor seeking a k-fold speedup: replicate the whole hardware k times, which also multiplies its cost, or divide the computation into k stages for a more modest cost increase. In the staged design, latches buffer intermediate values between stages so one stage can hold its result until the next stage accepts it.

In a synchronous k-stage pipeline, each stage is typically a combinational circuit and master–slave flip-flops act as the inter-stage latches. The latches isolate each stage's inputs from its outputs; when the clock is applied, they transfer data to the next stage simultaneously.

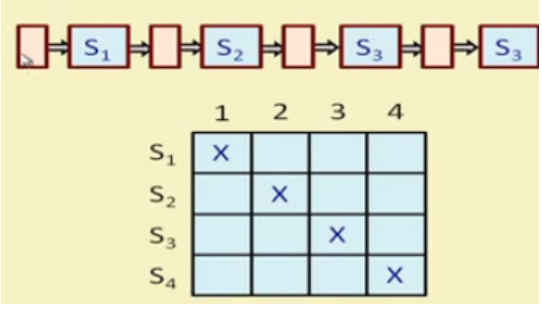


A linear pipeline executes its stages in order, such as A, B, C. A non-linear pipeline can use a different order or revisit a stage for the same data set; one possible sequence is A, B, C, B, C, A, C, A.

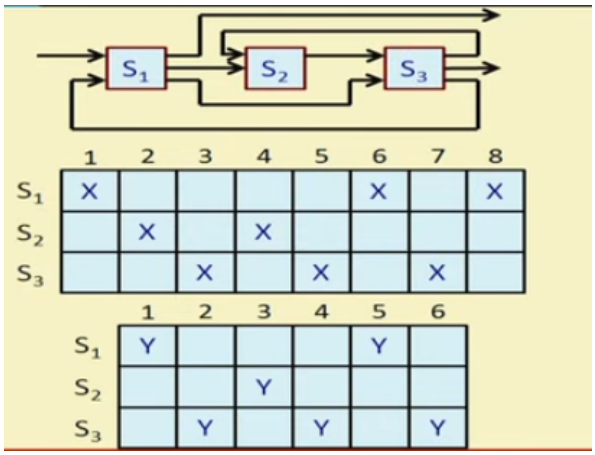


1.3 Reservation tables

A reservation table records which stages are used during each time step. It helps reveal repeated use of a stage, consecutive use of the same stage, and cases in which multiple stages are active in parallel. These patterns are useful for identifying potential resource conflicts. The horizontal axis marks time steps and the vertical axis marks stages; each column indicates the pipeline's activity at one time step. The number of columns gives the total evaluation time shown. A four-stage linear reservation table places one use in each successive stage.



For a three-stage dynamic, multifunction pipeline, the table can represent feedforward and feedback paths and separate functions (marked X and Y). Multiple X marks in one row show that a stage is reused in different cycles; adjacent X marks in a row show that it is used over consecutive cycles; and multiple X marks in one column show that several stages are used in parallel during one cycle.



1.4 Speedup, efficiency, and timing

Let t_i be the combinational delay in stage S_i. The slowest stage delay is

$$\tau_m = \max_{1 \le i \le k} t_i.$$

If an inter-stage latch contributes delay d_L, a first-order estimate of the pipeline clock period is $\tau = \tau_m + d_L$, with frequency $f = 1/\tau$. The clock period used below must also satisfy the physical timing constraint given later in this subsection.

For the performance comparison, assume a balanced k-stage pipeline accepts one input every cycle and does not stall. The first input requires k cycles to reach the output. Each of the remaining $N - 1$ inputs then requires one additional cycle, so the total number of cycles is

$$C_{\text{pipe}} = k + (N - 1), \qquad T_{\text{pipe}} = [k + (N - 1)]\tau.$$

Without pipelining, the same k stage operations are completed serially for each input. If each stage requires one time unit τ, one input takes $k\tau$; all N inputs therefore take

$$T_{\text{serial}} = N(k\tau) = Nk\tau.$$

This idealized reference processor ignores latch overhead. Speedup is the serial execution time divided by the pipelined execution time:

$$\begin{aligned} S_k &= \frac{T_{\text{serial}}}{T_{\text{pipe}}} = \frac{Nk\tau}{[k + (N - 1)]\tau} \\ &= \frac{Nk}{k + (N - 1)} = \frac{k}{1 + (k - 1)/N}. \end{aligned}$$

The last form makes the large-input limit explicit:

$$\lim_{N \to \infty} S_k = k.$$

For $k > 1$ and finite N, filling the pipeline adds $k - 1$ cycles beyond the N-cycle output schedule, so the speedup is below the ideal value k.

Efficiency is the achieved speedup divided by the ideal k-fold speedup:

$$E_k = \frac{S_k}{k} = \frac{N}{k + (N-1)}.$$

Thus $E_k \leq 1$ and $E_k \to 1$ as N grows. Average throughput over the full run is the number of completed inputs divided by the total time:

$$H_{k,N} = \frac{N}{T_{\text{pipe}}} = \frac{N}{[k + (N-1)]\tau}.$$

After the pipeline fills, its steady-state throughput is one result per cycle, or $1/\tau$. For example, with $k = 4$ stages and $N = 8$ inputs, the pipeline uses 11 cycles while the serial processor uses 32; hence $S_4 = 32/11$ and $E_4 = 8/11$.

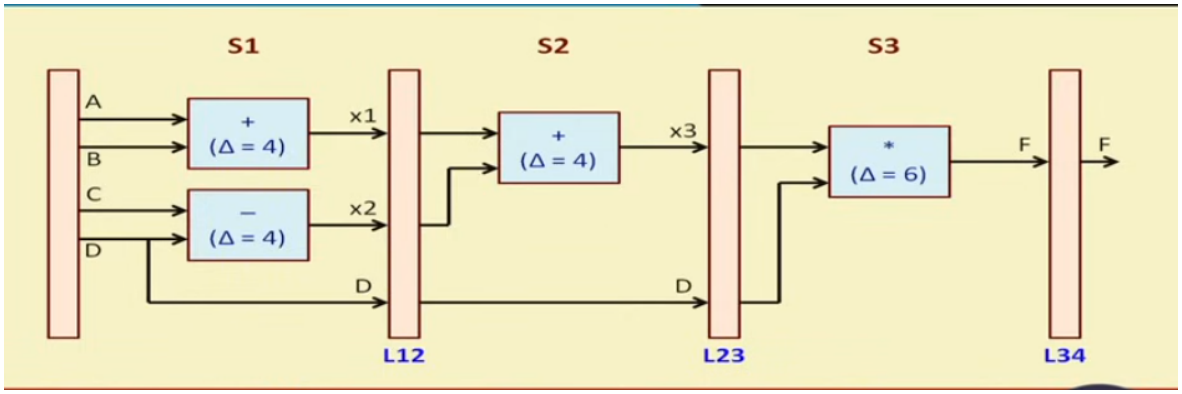
The chosen clock period must leave enough time for the slowest stage's logic, latch setup, and clock uncertainty:

$$\tau \geq t_{\text{logic,max}} + t_{\text{setup}} + t_{\text{skew}} + t_{\text{jitter}}, \qquad t_{\text{logic,max}} = \max_i t_i.$$

Clock skew is the difference in clock-arrival times between relevant latches; jitter is the variation in an edge's arrival time from cycle to cycle. Setup time is the interval for which data must be stable before the receiving latch captures it. A faster clock would violate this bound and could cause incorrect capture.

2 A Simple Pipeline Model

The example uses a three-stage computation with unsigned N-bit inputs A, B, C, and D, and an N-bit output F. Stage 1 computes $x_1 = A + B$ and $x_2 = C - D$; stage 2 computes $x_3 = x_1 + x_2$; and stage 3 computes $F = x_3 \times D$. Since D is used in both the first and third stages, its value must be forwarded through the intermediate registers to remain aligned with the corresponding sum.



The registers between stages preserve each cycle's intermediate values while the next input advances through the earlier stages. The following listing transcribes the original single-block Verilog model; its delay controls illustrate simulation timing.

2.1 One sequential block

```
module pipe_ex (F, A, B, C, D, clk);
  parameter N = 10;

  input [N-1:0] A, B, C, D;
  input clk;
  output [N-1:0] F;
  reg [N-1:0] L12_x1, L12_x2, L12_D, L23_x3, L23_D, L34_F;

  assign F = L34_F;

  always @(posedge clk)
    begin
      L12_x1 <= #4 A + B;
      L12_x2 <= #4 C - D;
      L12_D  <= D;                    // ** STAGE 1 **

      L23_x3 <= #4 L12_x1 + L12_x2;
      L23_D  <= L12_D;                // ** STAGE 2 **

      L34_F  <= #6 L23_x3 * L23_D;    // ** STAGE 3 **
    end
endmodule
```

Nonblocking assignments make all right-hand sides use the values present at the start of the clock event. The `#4` and `#6` delays are simulation delays, not a substitute for synthesizable timing constraints.

2.2 Separate block per stage

The same pipeline can be written with one clocked block per stage. This presentation makes the stage boundaries more explicit; the code is transcribed from the alternative version in the notes.

```
module pipe_ex (F, A, B, C, D, clk);
  parameter N = 10;

  input [N-1:0] A, B, C, D;
  input clk;
  output [N-1:0] F;
  reg [N-1:0] L12_x1, L12_x2, L12_D, L23_x3, L23_D, L34_F;

  assign F = L34_F;

  always @(posedge clk)               // ** STAGE 1 **
    begin
      L12_x1 <= #4 A + B;
      L12_x2 <= #4 C - D;
      L12_D  <= D;
    end
```

```
  always @(posedge clk)                // ** STAGE 2 **
    begin
      L23_x3 <= #4 L12_x1 + L12_x2;
      L23_D  <= L12_D;
    end

  always @(posedge clk)                // ** STAGE 3 **
    L34_F <= #6 L23_x3 * L23_D;
endmodule
```

2.3 Test bench

The test bench drives several input combinations, toggles the clock, and prints the output. The comments list the expected values from the original example.

```
module pipe1_test;
  parameter N = 10;
  wire [N-1:0] F;
  reg [N-1:0] A, B, C, D;
  reg clk;

  pipe_ex MYPIPE (F, A, B, C, D, clk);

  initial clk = 0;
  always #10 clk = ~clk;

  initial
    begin
      #5  A = 10; B = 12; C = 6;  D = 3;  // F = 75  (4Bh)
      #20 A = 10; B = 10; C = 5;  D = 3;  // F = 66  (42h)
      #20 A = 20; B = 11; C = 1;  D = 4;  // F = 112 (70h)
      #20 A = 15; B = 10; C = 8;  D = 2;  // F = 62  (3Eh)
      #20 A = 8;  B = 15; C = 5;  D = 0;  // F = 0   (00h)
      #20 A = 10; B = 20; C = 5;  D = 3;  // F = 66  (42h)
      #20 A = 10; B = 10; C = 30; D = 1;  // F = 49  (31h)
      #20 A = 30; B = 1;  C = 2;  D = 4;  // F = 116 (74h)
    end

  initial
    begin
      $dumpfile ("pipe1.vcd");
      $dumpvars (0, pipe1_test);
      $monitor ("Time: %d, F = %d", $time, F);
      #300 $finish;
    end
endmodule
```

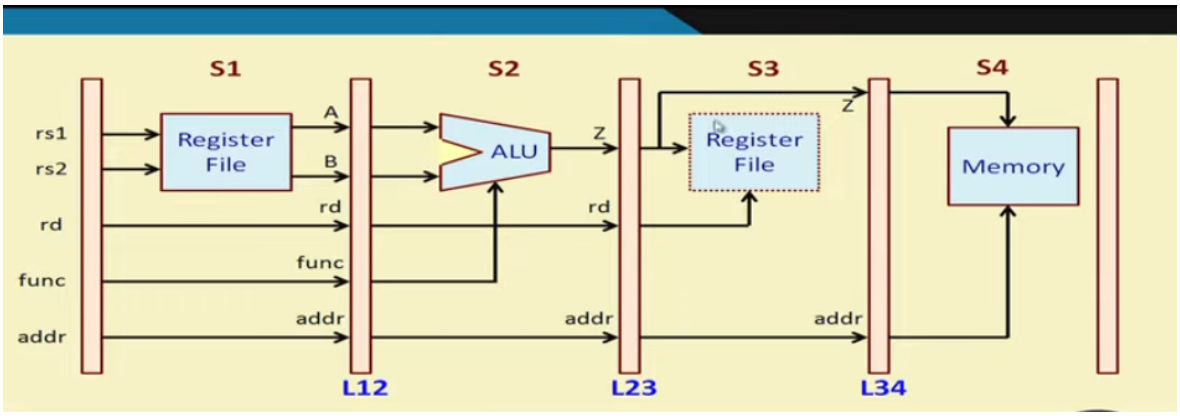
Clocking every inter-stage register from the same phase can cause race conditions in a physical pipeline. Master–slave flip-flops in the latches or a two-phase clock applied to alternate stages are possible remedies; the next example uses two non-overlapping phases.

3 A More Complex Pipeline

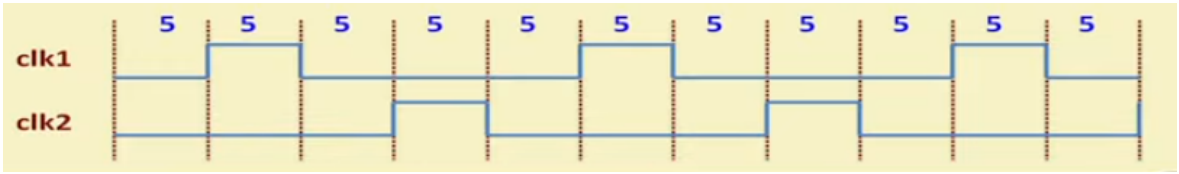
The four stages read two operands, perform an ALU operation, write the result to the destination register, and write it to memory. In stage 1, the 16-bit values addressed by `rs1` and `rs2` are read into A and B. Stage 2 applies the operation selected by `func` and stores the result in Z. Stage 3 writes Z to the register addressed by `rd`; stage 4 writes Z to memory at `addr`. The register bank contains sixteen 16-bit registers, addressed by the 4-bit fields `rs1`, `rs2`, and `rd`; two reads and one write are supported per clock cycle. The memory has 256 16-bit locations, addressed by `addr` using 8 bits, and a location can be read in one clock cycle. The address, destination register, and operation code travel through pipeline registers with the data.

The 4-bit `func` field selects the ALU operation:

`0000` ADD ($A + B$)	`0001` SUB ($A - B$)	`0010` MUL ($A \times B$)
`0011` SELA (A)	`0100` SELB (B)	`0101` AND ($A \,\&\, B$)
`0110` OR ($A \mid B$)	`0111` XOR ($A \oplus B$)	`1000` NEGA ($-A$)
`1001` NEGB ($-B$)	`1010` SRA (A right shift)	`1011` SLA (A left shift)



The diagram shows `clk1` and `clk2` offset so their active pulses do not overlap. Master–slave flip-flops or non-overlapping phases keep consecutive stages from capturing unstable data at the same time.



3.1 Four-stage module

This listing preserves the signal names and operations shown in the source screenshots. The register-bank read and ALU work use `clk1` and `clk2` on opposite edges as indicated by the example.

```
module pipe_ex2 (Zout, rs1, rs2, rd, func, addr, clk1, clk2);
  input [3:0] rs1, rs2, rd, func;
  input [7:0] addr;
  input write, clk1, clk2;
  output [15:0] Zout;

  reg [15:0] L12_A, L12_B, L23_Z, L34_Z;
  reg [3:0] L12_rd, L12_func, L23_rd;
  reg [7:0] L12_addr, L23_addr, L34_addr;

  reg [15:0] regbank [0:15];  // Register bank
  reg [15:0] mem [0:255];     // 256 x 16 memory

  assign Zout = L34_Z;

  always @(posedge clk1)
    begin
      L12_A    <= #2 regbank[rs1];
      L12_B    <= #2 regbank[rs2];
      L12_rd   <= #2 rd;
      L12_func <= #2 func;
      L12_addr <= #2 addr;              // ** STAGE 1 **
    end

  always @(negedge clk2)
    begin
      case (func)
        0:  L23_Z <= #2 L12_A + L12_B;
        1:  L23_Z <= #2 L12_A - L12_B;
        2:  L23_Z <= #2 L12_A * L12_B;
        3:  L23_Z <= #2 L12_A;
        4:  L23_Z <= #2 L12_B;
        5:  L23_Z <= #2 L12_A & L12_B;
        6:  L23_Z <= #2 L12_A | L12_B;
        7:  L23_Z <= #2 L12_A ^ L12_B;
        8:  L23_Z <= #2 -L12_A;
        9:  L23_Z <= #2 -L12_B;
        10: L23_Z <= #2 L12_A >> 1;
        11: L23_Z <= #2 L12_A << 1;
        default: L23_Z <= #2 16'hxxxx;
      endcase
      L23_rd   <= #2 L12_rd;
      L23_addr <= #2 L12_addr;          // ** STAGE 2 **
    end

  always @(posedge clk1)
    begin
      regbank[L23_rd] <= #2 L23_Z;
      L34_Z           <= #2 L23_Z;
      L34_addr        <= #2 L23_addr;   // ** STAGE 3 **
    end

  always @(negedge clk2)
```

```
    begin
      mem[L34_addr] <= #2 L34_Z;        // ** STAGE 4 **
    end
endmodule
```

The transcription follows the screenshot, including `case (func)` even though the operation code is also captured in `L12_func`. Confirm the intended signal before using this as a design implementation. The screenshot declares `write` even though it is omitted from the port list and unused in the shown module.

3.2 Test bench

The second test bench generates two clock phases, initializes the register bank, applies ALU operations, and prints the affected memory locations.

```
module pipe2_test;
  wire [15:0] Z;
  reg [3:0] rs1, rs2, rd, func;
  reg [7:0] addr;
  reg clk1, clk2;
  integer k;

  pipe_ex2 MYPIPE (Z, rs1, rs2, rd, func, addr, clk1, clk2);

  initial
    begin
      clk1 = 0; clk2 = 0;
      repeat (20)
        begin
          #5 clk1 = 1; #5 clk1 = 0;
          #5 clk2 = 1; #5 clk2 = 0;
        end
    end

  initial
    for (k = 0; k < 16; k = k + 1)
      MYPIPE.regbank[k] = k;            // Initialize registers

  initial
    begin
      #5  rs1 = 3;  rs2 = 5;  rd = 10; func = 0;  addr = 125; // ADD
      #20 rs1 = 3;  rs2 = 8;  rd = 12; func = 2;  addr = 126; // MUL
      #20 rs1 = 10; rs2 = 5;  rd = 14; func = 1;  addr = 128; // SUB
      #20 rs1 = 7;  rs2 = 3;  rd = 13; func = 11; addr = 127; // SLA
      #20 rs1 = 10; rs2 = 5;  rd = 15; func = 1;  addr = 129; // SUB
      #20 rs1 = 12; rs2 = 13; rd = 16; func = 0;  addr = 130; // ADD

      #60 for (k = 125; k < 131; k = k + 1)
        $display ("Mem[%3d] = %3d", k, MYPIPE.mem[k]);
    end
```

```
  initial
    begin
      $dumpfile ("pipe2.vcd");
      $dumpvars (0, pipe2_test);
      $monitor ("Time: %3d, F = %3d", $time, Z);
      #300 $finish;
    end
endmodule
```

The listing keeps the test values as printed in the source, including the destination value `rd = 16`; with a 4-bit ‘rd‘, that value does not identify a valid entry in the 16-register bank.